Week 4: Machine Learning – Deep Learning

Python 200 | Mentor Session

What is Deep Learning?

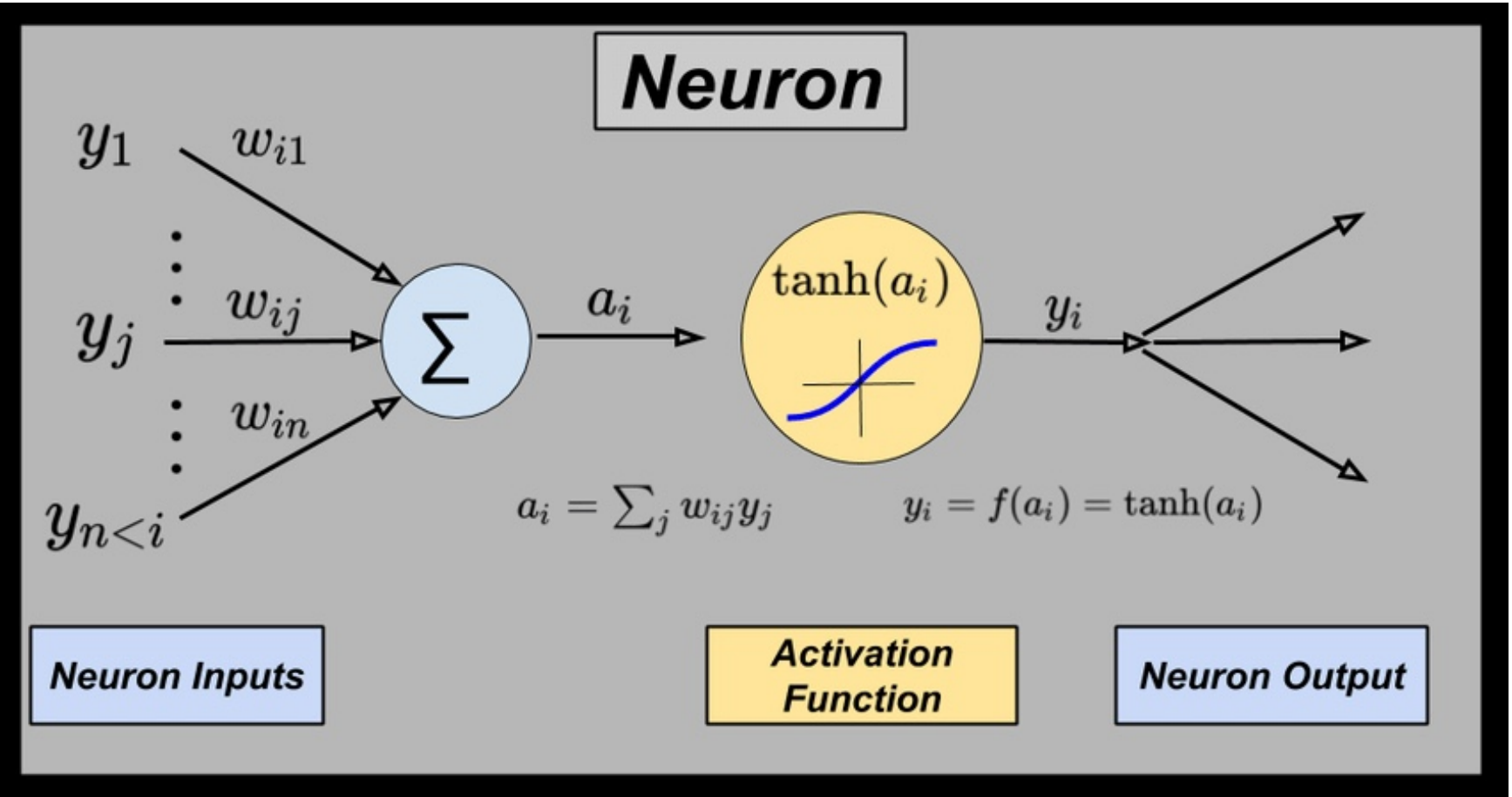
Concept	Real-World Examples
Deep learning = ML where models **learn their own internal representations** from data – no hand-crafted rules	ChatGPT → learns patterns in language
Built from **layers of neurons**, each transforming data a bit further	DALL-E → generates images from text descriptions
Loosely inspired by biology, but these are math – not brain simulations	Face unlock on your phone → classifies faces in real time
Goal this week: **build intuition**, not memorize equations	Medical imaging → detects tumors in scans

Ask students: What AI tools have you used recently? What do you think is happening under the hood?

Inside a Neuron

Concept	How It Works
A neuron does three simple things: 1. Takes in numbers (inputs) 2. Multiplies each by a **weight** (learnable) 3. Passes the result through an **activation function**	Without activation functions, even 100 layers would behave like a single straight line – no complex patterns
Weights are what the network learns	Think of weights as dials: training turns them to make better predictions
Activation function adds nonlinearity – allows learning curves and complex boundaries	Common choices: ReLU, sigmoid, tanh

See next slide for diagram →

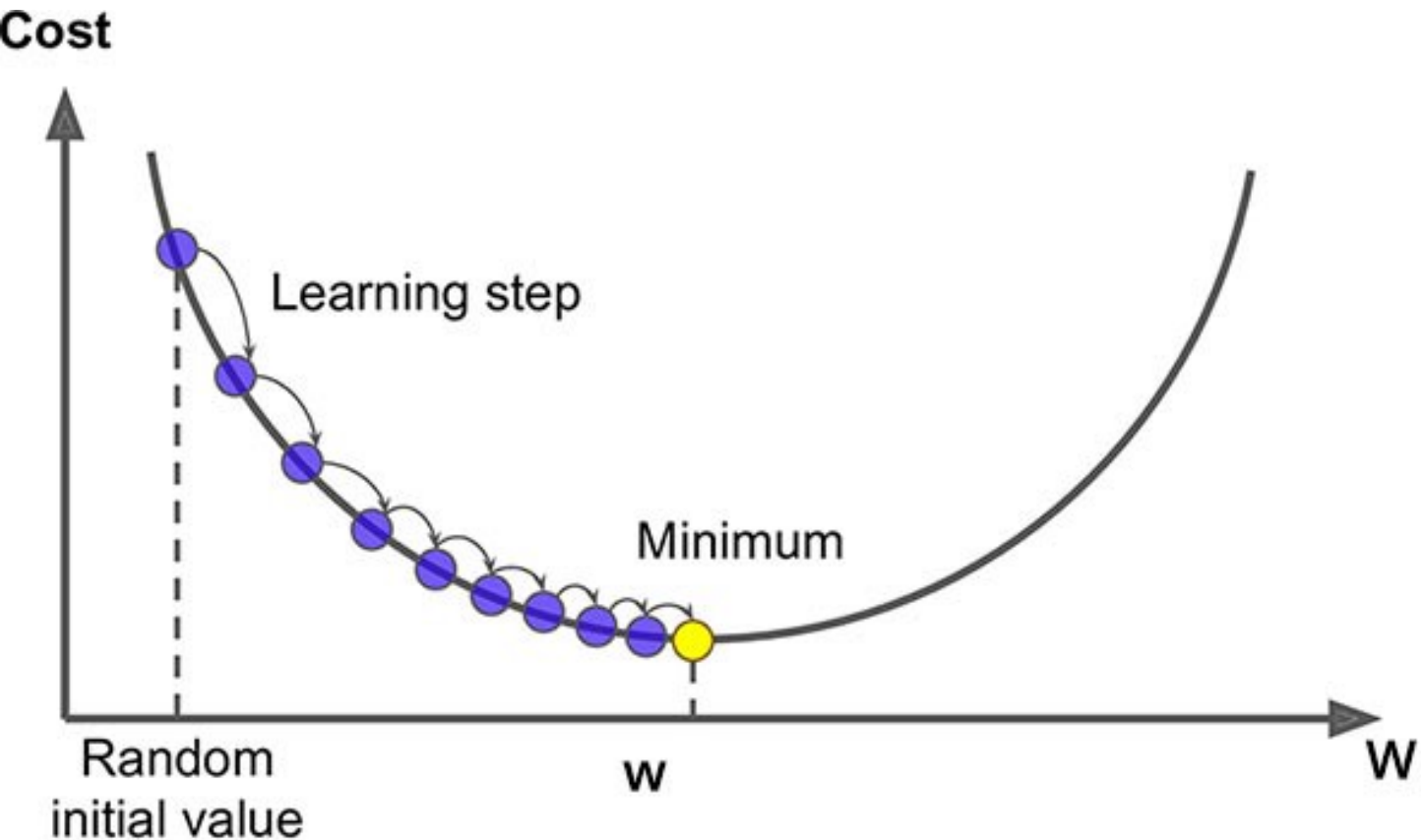

Neuron
y_1
w_{i1}
y_j
w_{ij}
w_{in}
$y_{n<i}$
$\sum$
a_i
$\tanh(a_i)$
y_i
$a_i = \sum_j w_{ij} y_j$
$y_i = f(a_i) = \tanh(a_i)$
Neuron Inputs
Activation Function
Neuron Output

How Networks Learn: Backpropagation

Step	Analogy
Forward pass: data flows through all layers → prediction produced	Roll a ball down a slope
Compare prediction to true answer → calculate **loss** (error)	Measure how far it landed from the target
Backward pass: error flows backward, each weight adjusted slightly to reduce loss	Adjust the launch angle a little
Repeat thousands of times → network gradually improves	After many tries, you find the best angle

This is **gradient descent** – following the slope of the error surface downhill toward lower loss. *See next slide* →

> **Ask students:** What do you think happens if weight updates are too large? What about too small?

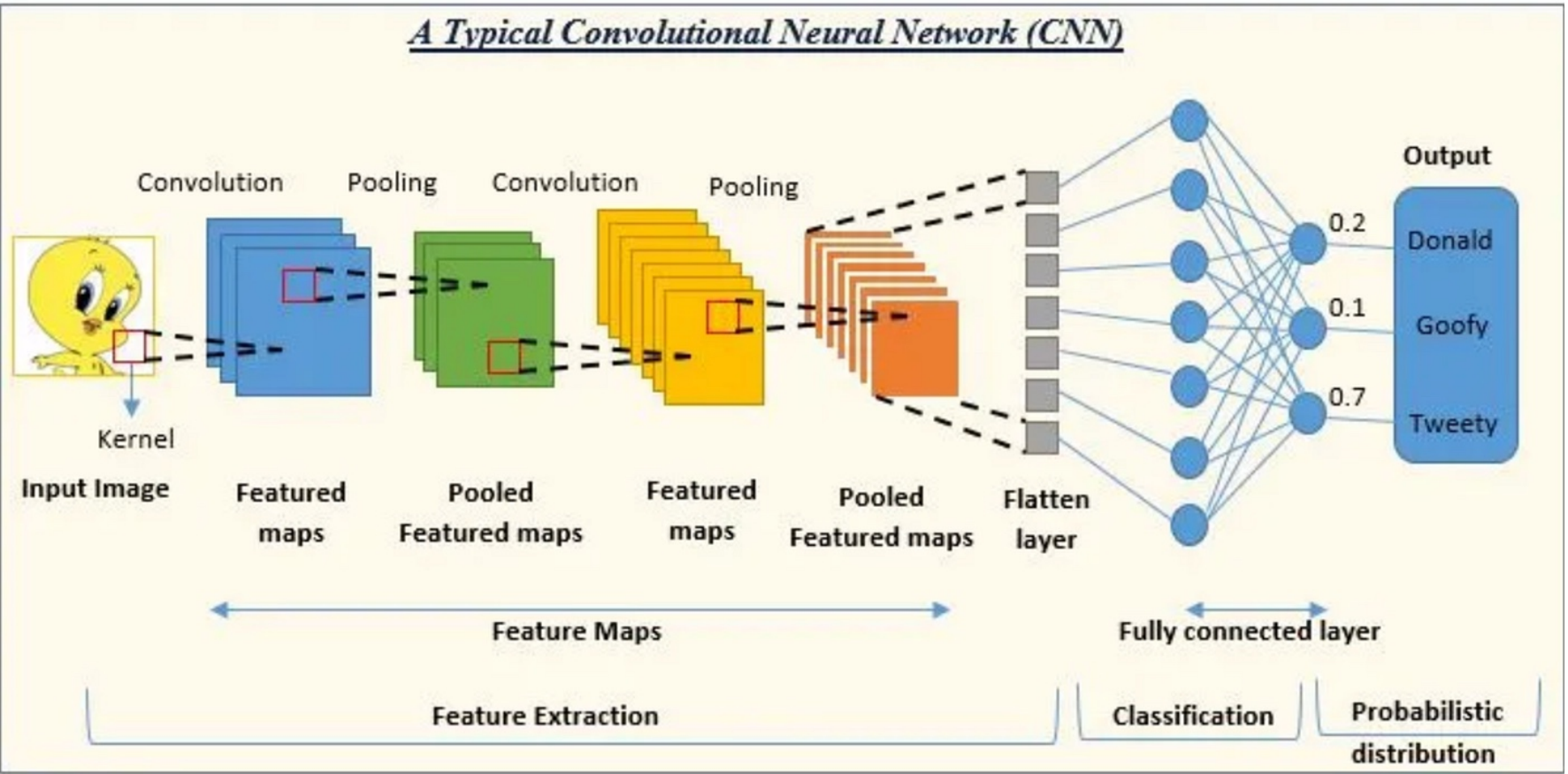

Cost
Learning step
Minimum
Random
initial value
w
W

The Deep Learning Revolution

Before 2012	After 2012
Neural networks had 1–2 layers	Deep networks have hundreds of layers
Training was slow — limited GPU support	GPUs made large-scale training practical
Required hand-crafted image features	Models learn their own features from raw pixels
Could only solve narrow, simple problems	Handles images, language, speech, video

2012: CNNs dramatically outperformed all previous image recognition methods → deep learning era begins.

CNNs learn hierarchically: edges → shapes → objects → scenes. *See next slide* →


A Typical Convolutional Neural Network (CNN)
Convolution
Pooling
Convolution
Pooling
Output
0.2
0.1
0.7
Donald
Goofy
Tweety
Kernel
Input Image
Featured maps
Pooled Featured maps
Featured maps
Pooled Featured maps
Flatten layer
Feature Maps
Fully connected layer
Feature Extraction
Classification
Probabilistic distribution

PyTorch – Your Deep Learning Toolkit

What It Is	What It Handles For You
Open-source framework, originally from Facebook AI	Layers and network building blocks
Used by OpenAI, Google DeepMind, most major AI labs	Automatic backpropagation
Intuitive, Pythonic design – comparatively easy to learn	Optimization algorithms (Adam, SGD, etc.)
Strong GPU support across all platforms	GPU acceleration + pretrained models

You specify: **architecture, loss function, optimizer**. PyTorch handles the rest.

> **Ask students:** Has anyone used NumPy before? PyTorch tensors are basically NumPy arrays that can run on a GPU.

Tensors – PyTorch's Core Data Structure

Concept	Code Example
A **tensor** is an array of numbers – like NumPy, but GPU-capable	`torch.tensor([1, 2, 3])` → 1D tensor (3 elements)
Key properties: **shape** and **dtype**	`torch.rand(3, 4)` → 3×4 matrix of random floats
All standard math operations work element-wise	`a + b`, `a * b`, `a.mean()`
Setting a **random seed** makes results reproducible	`torch.manual_seed(42)` before any random operation

```python
x = torch.rand(3, 4)
print(x.shape)   # torch.Size([3, 4])
print(x.dtype)   # torch.float32
```

Kaggle Demo: PyTorch Tensors *(Lesson 2)*

Open: Lesson 2 → "Open in Kaggle" button

Focus on	Skim for now
Creating tensors	Broadcasting rules
Random tensors and seeding	`requires_grad` / autograd
Shapes and dtypes	
Basic math operations	

Walk through notebook together, then give students time to experiment.

> **Ask students:** Run `torch.manual_seed(42)` then `torch.rand(3)` twice in a row. What do you notice? Why does reproducibility matter in ML?

The Training Workflow

Step	What's Happening
1. **Forward pass**	Feed data through the network → get prediction
2. **Calculate loss**	Compare prediction to true answer
3. **Zero gradients**	Clear leftover gradients from the previous step
4. **Backpropagation**	Compute how much each weight contributed to the error
5. **Update weights**	Optimizer adjusts weights to reduce loss

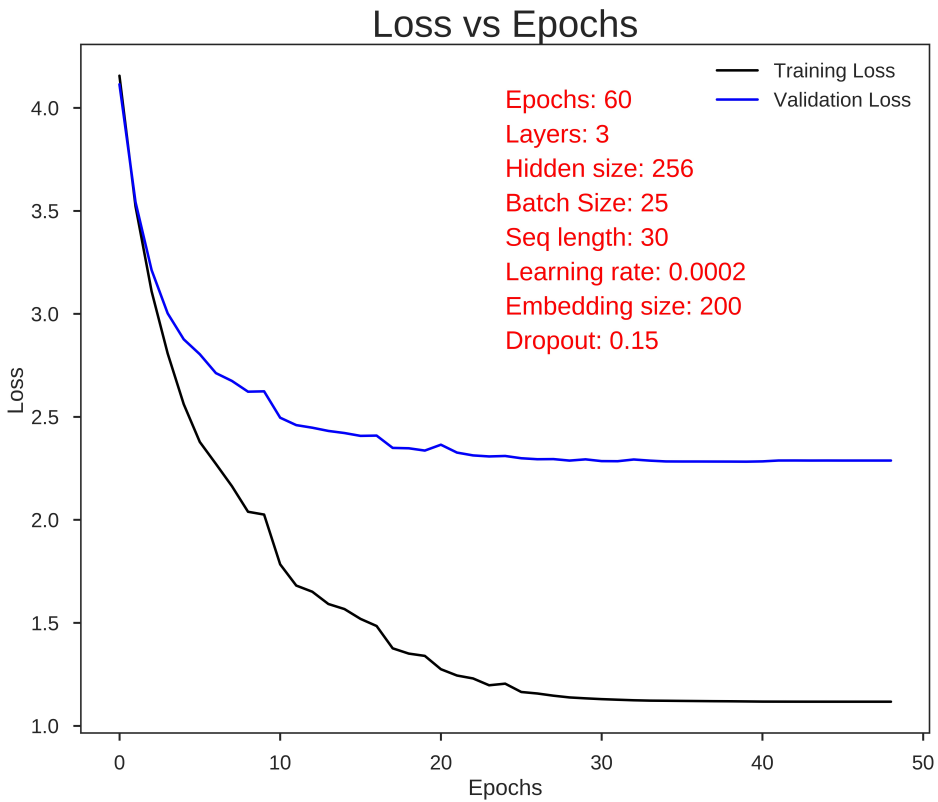
This loop runs for many **epochs** – each epoch is one full pass through the training data.

> **Ask students:** Why do we need to zero the gradients each step? What might happen if we didn't?

Loss, Optimizer, and Epochs

Concept	Example / Detail
Loss function: measures how wrong the model is	Cross-entropy loss for classification; MSE for regression
Optimizer: decides how to update weights	Adam is a strong default; SGD is simpler
Learning rate: size of each weight update step	Too high → unstable. Too low → very slow.
Epoch: one full pass through all training data	Training for 100 epochs = network sees all data 100 times

A good sign: **loss decreases** over epochs, **accuracy increases**. *See next slide* →


Loss vs Epochs
Training Loss
Validation Loss
Epochs: 60
Layers: 3
Hidden size: 256
Batch Size: 25
Seq length: 30
Learning rate: 0.0002
Embedding size: 200
Dropout: 0.15
Loss
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0
10
20
30
40
50
Epochs

Kaggle Demo: First Neural Network *(Lesson 3)*

Open: Lesson 3 → "Open in Kaggle" button

Important: Enable GPU before running → Settings → Accelerator → GPU T4 x2

Watch for	Why it matters
Loss going down over epochs	The network is learning
Decision boundary visualization	See what the model actually learned
Training vs. test accuracy	Are we overfitting?

The dataset: **concentric circles** – two classes in nested rings. A linear model can't separate them. A neural network can.

> **Ask students:** What would the decision boundary look like for a simpler model, like logistic regression? Why can't it solve this?

CNN Inference: Using Pretrained Models *(Lesson 4)*

Concept	In Practice
CNN = Convolutional Neural Network – designed for image data	Learns edges → textures → shapes → objects, layer by layer
Inference = using a trained model to predict on new data, with no training	We just load and run – weights are frozen
TorchVision provides CNNs pretrained on **ImageNet** (1.2M images, 1000 classes)	ResNet, VGG, EfficientNet all available out of the box
These models have already learned rich, general visual features	Ready to use in production pipelines immediately

Ask students: What's the advantage of using a pretrained model over training one from scratch?

Evaluation Mode and Image Preprocessing

Concept	Why It Matters
`model.eval()` switches to inference mode	Disables dropout and batch normalization's training behavior
Images must be preprocessed to match what the model expects	Wrong size or normalization → garbage predictions
Transform pipeline: resize → center crop → to tensor → normalize	Must match exactly what was used during the model's original training

```
model.eval()
with torch.no_grad():      # don't track gradients during inference
    output = model(image_tensor)
```

Ask students: Why would preprocessing images differently from training cause problems – even if the images themselves look fine?

Kaggle Demo: CNN Inference *(Lesson 4)*

Open: Lesson 4 → "Open in Kaggle" button

Watch for	Why it matters
Top-5 predicted classes with scores	CNNs output a probability for every class
High vs. low confidence predictions	How certain is the model?
Unusual or ambiguous images	Models can be confidently and silently wrong

Try different images – what does the model get right? What confuses it?

> **Ask students:** What's the difference between a model being wrong with low confidence vs. wrong with high confidence? In what settings does that distinction matter most?

Transfer Learning: Reusing Learned Features *(Lesson 5)*

Concept	Analogy
Take a model trained on one task, adapt it to a related task	A generalist doctor specializing in cardiology
Early CNN layers learn **general features** – edges, textures, shapes – useful for any image task	The foundational knowledge transfers
Replace only the final classification layer to match the new task	Swap just the specialization
Train on far less data, in far less time	Skip years of med school

Why it works: features learned from millions of images generalize broadly to new image tasks.

> **Ask students:** Can you think of a real use case where you'd adapt an existing model rather than train from scratch?

Feature Extraction vs. Fine-Tuning

Feature Extraction	Fine-Tuning
Freeze all pretrained layers	Unfreeze some or all pretrained layers
Only train the new classification head	Train classification head + some earlier layers
Faster, needs less data	More adapted to the new task
Best when new task is similar to original	Better when new task differs more

```
# Freeze all pretrained weights
for param in model.parameters():
    param.requires_grad = False

# Add new trainable head (unfrozen by default)
model.fc = nn.Linear(model.fc.in_features, num_classes)
```

Kaggle Demo: Transfer Learning *(Lesson 5)*

Open: Lesson 5 → "Open in Kaggle" button

Dataset: Fashion MNIST – 10 categories of clothing

Watch for	Why it matters
How quickly accuracy improves	Transfer learning converges much faster than training from scratch
Validation accuracy vs. training accuracy	Watch for overfitting
Final classification layer replacing the original	Only the head is being retrained

Ask students: We froze the early layers and only retrained the final layer. What would happen if we also unfroze the earlier layers and trained them? When might that be the right call?

Week 4 Wrap-Up

Topic	The Big Idea
Neural networks	Simple units combined → complex pattern learning
Backpropagation	Error flows backward, weights adjust, model improves
PyTorch tensors	GPU-powered NumPy – same ideas, new hardware
Training workflow	Forward → loss → backward → update, repeat
CNN inference	Load a pretrained model, run it immediately
Transfer learning	Adapt, don't rebuild from scratch

The real-world takeaway: most production deep learning pipelines are transfer learning pipelines. You rarely train from scratch.

> **Ask students:** Which of these topics would you want to go deeper on? What felt most unclear?